



UNIVERSIDADE FEDERAL DO PARÁ
INSTITUTO DE TECNOLOGIA FACULDADE DE ENGENHARIA DA
COMPUTAÇÃO E TELECOMUNICAÇÕES

AMANDA GABRIELLY PRESTES LOPES - 202207040043
BIANCA CRISTINA DOS SANTOS BRITO - 202006840016
GILTON CONCEICAO CAMINHA - 202106840008
JOAO LORRAN GUIMARAES MENEZES - 201706840040
YASMIM CARVALHO DA SILVA - 202007040025

**IMPLEMENTAÇÃO DE UMA CALCULADORA DE SISTEMAS DE
EQUAÇÕES LINEARES**

BELEM - PA
2025

AMANDA GABRIELLY PRESTES LOPES - 202207040043
BIANCA CRISTINA DOS SANTOS BRITO - 202006840016
GILTON CONCEICAO CAMINHA - 202106840008
JOAO LORRAN GUIMARAES MENEZES - 201706840040
YASMIM CARVALHO DA SILVA - 202007040025

IMPLEMENTAÇÃO DE UMA CALCULADORA DE SISTEMAS DE EQUAÇÕES LINEARES

Relatório da segunda atividade avaliativa da disciplina de Métodos Numéricos ministrada pelo professor Jeferson Danilo Lima Silva para o curso de Engenharia de Telecomunicações do Instituto de Tecnologia da Universidade Federal do Pará - Campus Belém.

BELÉM - PA
2025

Sumário

1	INTRODUÇÃO	4
1.1	Visão Geral	4
1.2	Objetivos	4
2	DESENVOLVIMENTO	5
2.1	Metodologia	5
2.2	Preparação e Redução à Forma Escalonada	5
2.3	Redução à Forma Canônica	7
2.4	Análise da Solução e Saída	8
3	Integralização e Teste	9
4	Resultados e Questões	10
4.1	Problema 2.2	10
4.2	Problema 2.3	11
4.3	Problema 2.4	11
4.4	Problema 2.1	12
5	Conclusão	14
	Referências	15

1 INTRODUÇÃO

1.1 Visão Geral

A teoria das equações lineares desempenha um papel importante e motivador no campo da Álgebra Linear. Muitos problemas na Álgebra Linear, como a busca pelo núcleo de uma transformação linear ou a caracterização do subespaço gerado por um conjunto de vetores, são equivalentes ao estudo de um sistema de equações lineares. Por simplicidade, as equações deste trabalho são sobre o corpo real $\mathbb{R}$. No entanto, os resultados e técnicas também se aplicam a equações sobre o corpo complexo $\mathbb{C}$ ou qualquer outro corpo arbitrário K . O trabalho aborda a solução de um sistema de equações lineares. (CHAPRA; CANALE, 2015)

1.2 Objetivos

O principal objetivo deste trabalho é desenvolver e implementar uma rotina para resolver sistemas de equações lineares de forma robusta e precisa, cobrindo os três casos possíveis de solução: solução única, infinitas soluções ou nenhuma solução. A implementação deve ser feita "do zero", sem depender de bibliotecas de terceiros para as operações elementares. A rotina deve ser capaz de reduzir uma matriz aumentada à sua forma escalonada e, em seguida, à forma canônica, a partir da qual a solução pode ser determinada e apresentada ao usuário. O programa final deve ser testado com exemplos que demonstrem sua capacidade de lidar com cada tipo de cenário de solução.

2 DESENVOLVIMENTO

2.1 Metodologia

A metodologia para esta tarefa baseia-se no processo de eliminação de Gauss-Jordan para resolver sistemas de equações lineares. A abordagem é dividida em várias etapas principais para garantir a robustez da solução, que deve ser capaz de lidar com os três possíveis cenários: solução única, infinitas soluções e ausência de solução.

2.2 Preparação e Redução à Forma Escalonada

O processo inicia com a leitura da matriz aumentada do sistema linear. Em seguida, a rotina de eliminação de Gauss é aplicada para reduzir a matriz à sua forma escalonada equivalente. Durante esta fase, são realizadas operações elementares com as linhas. Uma etapa crucial é o tratamento de pivôs nulos, onde, antes de qualquer operação de eliminação, a linha do pivô nulo é permutada com a primeira linha abaixo que possua um valor não nulo na mesma coluna. Se todos os elementos abaixo do pivô forem nulos, o programa avança para a próxima coluna. A rotina otimiza o processo ao pular a eliminação de elementos que já são zero. Caso uma linha inconsistente ($0x_1 + \dots + 0x_n = b$, com $b \neq 0$) seja detectada, o processo é interrompido, e o sistema é classificado como inconsistente. Ao final desta etapa, as linhas nulas são removidas, e a matriz escalonada é apresentada ao usuário.

Segue o trecho do código:

```

1 def escalonar(matriz, m, n):
2     """
3     Reduz a matriz      forma escalonada.
4     """
5     matriz_temp = [list(linha) for linha in matriz] # Cria uma c
6     ↪ópia para não modificar a original
7     linha_pivo = 0
8
9     for coluna in range(n):
10        if linha_pivo >= m:
11            break
12
13        # 2.1) Encontrar pivô não nulo e trocar as linhas
14        if matriz_temp[linha_pivo][coluna] == 0:
15            encontrar_linha_nao_nula = -1
16            for i in range(linha_pivo + 1, m):
17                if matriz_temp[i][coluna] != 0:
18                    encontrar_linha_nao_nula = i

```

```

18         break
19
20     if encontrar_linha_nao_nula != -1:
21         trocar_linhas(matriz_temp, linha_pivo,
22             ↳ encontrar_linha_nao_nula)
23     else:
24         continue # Pular para a próxima coluna se todos
25             ↳ os elementos abaixo forem zero
26
27
28     if matriz_temp[linha_pivo][coluna] == 0:
29         continue
30
31     # 2.2) e 2.5) Eliminar elementos abaixo do pivô
32     for i in range(linha_pivo + 1, m):
33         if matriz_temp[i][coluna] != 0:
34             fator = matriz_temp[i][coluna] / matriz_temp[
35                 ↳ linha_pivo][coluna]
36             for j in range(coluna, n + 1):
37                 matriz_temp[i][j] = matriz_temp[i][j] - fator
38                 ↳ * matriz_temp[linha_pivo][j]
39
40     # 2.3) Verificar inconsistência
41     # Se um pivô se tornou zero durante a eliminação e o
42     ↳ termo constante não é
43     # zero, o sistema é inconsistente.
44     if matriz_temp[linha_pivo][coluna] == 0 and matriz_temp[
45         ↳ linha_pivo][n] != 0:
46         print("Sistema_inconsistente_detectado!")
47         return None # Retorna None para indicar que não há
48             ↳ solução
49
50     print(f"Matriz_após_processar_coluna_{coluna}:")
51     imprimir_matriz(matriz_temp)
52
53     linha_pivo += 1
54
55     # 2.4) Remover linhas nulas
56     matriz_limpa = [linha for linha in matriz_temp if any(abs(x)
57         ↳ > 1e-9 for x in linha)]
58
59     print("Matriz_escalonada_final:")

```

```

51     imprimir_matriz(matriz_limpa)
52
53     return matriz_limpa

```

Listing 1: Trecho do Código def escalonar

2.3 Redução à Forma Canônica

A partir da matriz escalonada, a metodologia continua com a eliminação de Gauss-Jordan, operando de baixo para cima. Nesta fase, os elementos acima dos pivôs são anulados, resultando na forma canônica da matriz.

Segue o trecho do código:

```

1  def canonizar(matriz_escalonada, m, n):
2      """
3      Reduz a matriz escalonada para a forma canônica.
4      """
5      r = len(matriz_escalonada)
6      matriz_canonizada = [list(linha) for linha in
7                             ↪matriz_escalonada]
8
9      for i in range(r - 1, -1, -1):
10         pivo_j = -1
11         for j in range(n):
12             if abs(matriz_canonizada[i][j]) > 1e-9:
13                 pivo_j = j
14                 break
15
16         if pivo_j != -1:
17             # Normalizar o pivô para 1
18             fator_normalizacao = matriz_canonizada[i][pivo_j]
19             for j in range(pivo_j, n + 1):
20                 matriz_canonizada[i][j] /= fator_normalizacao
21
22             # Anular elementos acima do pivô
23             for k in range(i - 1, -1, -1):
24                 fator Eliminacao = matriz_canonizada[k][pivo_j]
25                 for j in range(pivo_j, n + 1):
26                     matriz_canonizada[k][j] -= fator_elimizacao *
27                         ↪matriz_canonizada[i][j]
28
29         print("Matriz na forma canônica:")

```



```

28     imprimir_matriz(matriz_canonizada)
29     return matriz_canonizada

```

Listing 2: Trecho do Código def canonizar

2.4 Análise da Solução e Saída

A última etapa consiste na análise da matriz em sua forma canônica. Se o número de linhas não nulas (r) for igual ao número de incógnitas (n), o sistema possui uma única solução, que é extraída diretamente dos termos constantes da matriz.

Se r for menor que n , o sistema tem infinitas soluções, e a rotina identifica as $n - r$ variáveis livres, apresentando a solução geral em função delas.

A metodologia foi validada usando exemplos do material de apoio que cobrem todos os cenários de solução. Segue o trecho do código:

```

1  def analisar_solucão(matriz_canonizada, n):
2      """
3      Analisa a matriz na forma canônica e imprime a solução.
4      """
5      r = len(matriz_canonizada)
6
7      if r == n:
8          print("O sistema tem uma única solução.")
9          solucao = [linha[n] for linha in matriz_canonizada]
10         print(f"Solução: {solucao}")
11     else:
12         print("O sistema tem infinitas soluções.")
13         variaveis_livres = n - r
14         print(f"Quantidade de variáveis livres: {variaveis_livres}
15               ↪")
16
17         pivos = [next((j for j, val in enumerate(linha) if abs(
18             ↪val) > 1e-9), -1) for linha in matriz_canonizada]
19
20         solucao_geral = [""] * n
21         letras = [f"t{i+1}" for i in range(variaveis_livres)]
22         letra_idx = 0
23
24         variavel_is_pivo = [False] * n
25         for p in pivos:
26             if p != -1:
27                 variavel_is_pivo[p] = True

```

```

26
27     for j in range(n):
28         if not variavel_is_pivo[j]:
29             solucao_geral[j] = letras[letra_idx]
30             letra_idx += 1
31
32     for i in range(r):
33         pivo_j = pivos[i]
34         expressao = f"{matriz_canonizada[i][n]:.2f}"
35         for j in range(pivo_j + 1, n):
36             if abs(matriz_canonizada[i][j]) > 1e-9:
37                 if matriz_canonizada[i][j] < 0:
38                     expressao += f"_{-matriz_canonizada[i][j]:.2f}*{solucao_geral[j]}"
39                 else:
40                     expressao += f"_{matriz_canonizada[i][j]:.2f}*{solucao_geral[j]}"
41         solucao_geral[pivo_j] = expressao
42
43     print("Solução_geral:")
44     for idx, val in enumerate(solucao_geral):
45         print(f"x{idx+1}_{val}")

```

Listing 3: Trecho do Código def *analisa_solucao*

3 Integralização e Teste

O código foi estruturado em funções modulares, cada uma com uma responsabilidade específica:

- `imprimir_matriz(matriz)`: Função utilitária que formata e exibe a matriz na tela, facilitando a visualização das transformações em cada etapa.
- `trocar_linhas(matriz, l1, l2)`: Função auxiliar para permutar duas linhas da matriz, um passo fundamental no processo de tratamento de pivôs nulos.
- `escalonar(matriz, m, n)`: Esta é a principal função de eliminação de Gauss. Ela itera sobre as colunas para criar os pivôs e anular os elementos abaixo deles. A função incorpora a lógica de busca e permuta de pivôs não nulos e a otimização para pular elementos já zerados. O retorno

None é utilizado para sinalizar um sistema inconsistente, interrompendo o fluxo do programa.

- `canonizar(matriz_escalonada, m, n)`: Esta função implementa a segunda fase da eliminação de Gauss-Jordan. Ela itera de baixo para cima, normaliza cada pivô para 1 e anula os elementos acima dele, transformando a matriz escalonada em sua forma canônica.
- `analisar_solucao(matriz_canonizada, n)`: Esta função analisa o resultado final. Ela determina o número de linhas não nulas da matriz canônica e o compara com o número de incógnitas. Com base nessa comparação, ela imprime se a solução é única ou se existem infinitas soluções, fornecendo a solução geral em termos das variáveis livres, quando aplicável.

O código-fonte completo deste projeto, incluindo a implementação dos métodos numéricos e a interface de integração, está disponível publicamente no repositório GitHub em atividade 2:

<https://github.com/a-mand/metodosN>

4 Resultados e Questões

4.1 Problema 2.2

Inicialmente, o código apresentou uma falha lógica na detecção de sistemas inconsistentes. O problema pode ser resumido da seguinte forma:

- A verificação de inconsistência era realizada apenas quando a linha do pivô atual se tornava totalmente nula, uma situação que não ocorria devido à garantia de que o pivô era não-nulo antes da eliminação.
- A lógica não cobria o caso em que uma linha inteira, que não era a linha do pivô, se tornava $[0, 0, \dots, 0, b]$ (com $b \neq 0$) após a conclusão das operações de eliminação para uma coluna.

O Problema 2.2, que envolve um sistema com 3 equações e 3 incógnitas, demonstra um cenário de sistema inconsistente. O processo de eliminação de Gauss, aplicado à matriz aumentada do sistema, revela que não há solução. Na forma escalonada, a rotina encontra uma linha de zeros nos coeficientes que corresponde a um valor não nulo na coluna do termo constante. Isso leva a uma contradição matemática, como $0 = -3$, o que prova que não há valores para as incógnitas que possam satisfazer todas as equações simultaneamente.

```

Digite o número de equações (m): 3
Digite o número de incógnitas (n): 3
Digite os elementos da matriz aumentada (linha por linha):
Linha 1: 1 2 -3 -1
Linha 2: 3 -1 2 7
Linha 3: 5 3 -4 2
Matriz após processar coluna 0:
  1.00   2.00  -3.00  -1.00
  0.00  -7.00  11.00  10.00
  0.00  -7.00  11.00   7.00
-----
Matriz após processar coluna 1:
  1.00   2.00  -3.00  -1.00
  0.00  -7.00  11.00  10.00
  0.00   0.00   0.00  -3.00
-----
Matriz escalonada final:
  1.00   2.00  -3.00  -1.00
  0.00  -7.00  11.00  10.00
  0.00   0.00   0.00  -3.00
-----
Matriz na forma canônica:
  1.00   0.00   0.14   1.86
  0.00   1.00  -1.57  -1.43
  0.00   0.00   0.00  -3.00
-----
O sistema tem uma única solução.
Solução: [1.8571428571428572, -1.4285714285714286, -3.0]

```

Figura 1: Questão 2.2 (Erro no código)

```

Matriz após processar coluna 0:
  1.00   2.00  -3.00  -1.00
  0.00  -7.00  11.00  10.00
  0.00  -7.00  11.00   7.00
-----
Matriz após processar coluna 1:
  1.00   2.00  -3.00  -1.00
  0.00  -7.00  11.00  10.00
  0.00   0.00   0.00  -3.00
-----
Sistema inconsistente detectado!

```

Figura 2: Questão 2.2 (Corrigido)

4.2 Problema 2.3

O Problema 2.3 é composto por 3 equações e 3 incógnitas. A rotina de eliminação de Gauss é aplicada para reduzir a matriz aumentada à sua forma escalonada. Em seguida, o processo de retro-substituição (parte da eliminação de Gauss-Jordan) é utilizado para obter a forma canônica da matriz. Como o número de equações com pivôs é igual ao número de incógnitas, o programa determina que há uma única solução para o sistema. A solução específica, extraída diretamente da matriz na forma canônica, é $x_1 = 1.0$, $x_2 = 2.0$ e $x_3 = -3.0$.

4.3 Problema 2.4

O Problema 2.4, que envolve um sistema com 3 equações e 3 incógnitas, demonstra um cenário de infinitas soluções. O processo de eliminação de Gauss é aplicado à matriz aumentada do sistema, que resulta em uma matriz escalonada com apenas duas linhas

```

Digite o número de equações (m): 3
Digite o número de incógnitas (n): 3
Digite os elementos da matriz aumentada (linha por linha):
Linha 1: 2 1 -2 10
Linha 2: 3 2 2 1
Linha 3: 5 4 3 4
Matriz após processar coluna 0:
2.00  1.00 -2.00  10.00
0.00  0.50  5.00 -14.00
0.00  1.50  8.00 -21.00
-----
Matriz após processar coluna 1:
2.00  1.00 -2.00  10.00
0.00  0.50  5.00 -14.00
0.00  0.00 -7.00  21.00
-----
Matriz após processar coluna 2:
2.00  1.00 -2.00  10.00
0.00  0.50  5.00 -14.00
0.00  0.00 -7.00  21.00
-----
Matriz escalonada final:
2.00  1.00 -2.00  10.00
0.00  0.50  5.00 -14.00
0.00  0.00 -7.00  21.00
-----
Matriz na forma canônica:
1.00  0.00  0.00  1.00
0.00  1.00  0.00  2.00
0.00  0.00  1.00 -3.00
-----
O sistema tem uma única solução.
Solução: [1.0, 2.0, -3.0]

```

Figura 3: Questão 2.3

```

Digite o número de equações (m): 3
Digite o número de incógnitas (n): 3
Digite os elementos da matriz aumentada (linha por linha):
Linha 1: 1 2 -3 6
Linha 2: 2 -1 4 2
Linha 3: 4 3 -2 14
Matriz após processar coluna 0:
1.00  2.00 -3.00  6.00
0.00 -5.00 10.00 -10.00
0.00 -5.00 10.00 -10.00
-----
Matriz após processar coluna 1:
1.00  2.00 -3.00  6.00
0.00 -5.00 10.00 -10.00
0.00  0.00  0.00  0.00
-----
Matriz escalonada final:
1.00  2.00 -3.00  6.00
0.00 -5.00 10.00 -10.00
-----
Matriz na forma canônica:
1.00  0.00  1.00  2.00
0.00  1.00 -2.00  2.00
-----
O sistema tem infinitas soluções.
Quantidade de variáveis livres: 1
Solução geral:
x1 = 2.00 - 1.00*t1
x2 = 2.00 + 2.00*t1
x3 = t1

```

Figura 4: Questão 2.4

não-nulas. Isso indica que há menos equações independentes do que incógnitas. O programa, ao reduzir a matriz para sua forma canônica, determina que há uma variável livre, representada por t_1 , pois o número de incógnitas (3) é maior que o número de equações com pivôs (2). A solução geral é apresentada, onde as variáveis x_1 e x_2 são expressas em termos de x_3 (a variável livre), confirmando a existência de uma infinidade de soluções para o sistema.

4.4 Problema 2.1

O Problema 2.1, conforme exibido na imagem, apresenta a resolução de um sistema de equações lineares com 3 equações e 5 incógnitas. A partir da matriz aumentada inicial, o programa executa uma série de operações para reduzir a matriz à forma escalonada e,

```

Digite o número de equações (m): 3
Digite o número de incógnitas (n): 5
Digite os elementos da matriz aumentada (linha por linha):
Linha 1: 2 -3 6 2 -5 3
Linha 2: 0 1 -4 1 0 1
Linha 3: 0 0 0 1 -3 2
Matriz após processar coluna 0:
  2.00  -3.00  6.00  2.00  -5.00  3.00
  0.00   1.00 -4.00  1.00   0.00  1.00
  0.00   0.00  0.00  1.00  -3.00  2.00
-----
Matriz após processar coluna 1:
  2.00  -3.00  6.00  2.00  -5.00  3.00
  0.00   1.00 -4.00  1.00   0.00  1.00
  0.00   0.00  0.00  1.00  -3.00  2.00
-----
Matriz após processar coluna 3:
  2.00  -3.00  6.00  2.00  -5.00  3.00
  0.00   1.00 -4.00  1.00   0.00  1.00
  0.00   0.00  0.00  1.00  -3.00  2.00
-----
Matriz escalonada final:
  2.00  -3.00  6.00  2.00  -5.00  3.00
  0.00   1.00 -4.00  1.00   0.00  1.00
  0.00   0.00  0.00  1.00  -3.00  2.00
-----
Matriz na forma canônica:
  1.00   0.00 -3.00  0.00   5.00 -2.00
  0.00   1.00 -4.00  0.00   3.00 -1.00
  0.00   0.00  0.00  1.00  -3.00  2.00
-----
O sistema tem infinitas soluções.
Quantidade de variáveis livres: 2
Solução geral:
x1 = -2.00 + 3.00*t1 - 5.00*t2
x2 = -1.00 + 4.00*t1 - 3.00*t2
x3 = t1
x4 = 2.00 + 3.00*t2
x5 = t2

```

Figura 5: Questão 2.1

em seguida, à forma canônica. O resultado final demonstra que o sistema tem infinitas soluções, com 2 variáveis livres. A solução geral é fornecida, expressando as variáveis x_1 , x_2 e x_4 em função das variáveis livres x_3 e x_5 , que são representadas por t_1 e t_2 .

5 Conclusão

A implementação da calculadora de sistemas de equações lineares demonstrou ser uma ferramenta eficaz e robusta para a resolução de problemas no campo da Álgebra Linear. Através da metodologia de eliminação de Gauss-Jordan, a rotina desenvolvida foi capaz de processar e analisar sistemas lineares de forma precisa, identificando os três cenários possíveis de solução.

Os testes sistemáticos com os exemplos do material de apoio confirmaram a validade da abordagem. O programa foi capaz de detectar corretamente um sistema inconsistente (Problema 2.2), encontrar a solução única de um sistema bem-determinado (Problema 2.3) e determinar a existência de infinitas soluções, calculando a quantidade de variáveis livres e expressando a solução geral de forma precisa (Problemas 2.4 e 2.1).

A construção da rotina "do zero", sem o uso de bibliotecas de terceiros para as operações elementares, não apenas atendeu às restrições do problema, mas também aprofundou a compreensão dos princípios matemáticos subjacentes. A capacidade do programa de lidar com diferentes cenários de solução e de apresentar o resultado de forma clara e estruturada valida o sucesso da implementação e a eficácia da metodologia aplicada.

Referências

CHAPRA, S. C.; CANALE, R. P. **Métodos Numéricos para Engenharia**. 7. ed. Porto Alegre: AMGH, 2015.